📚 StickerNest Terminology & Definitions

A living reference for all core concepts in StickerNest. These definitions are intentionally kept precise so that architecture, code, and conversation all use the same language.

Architectural Model

StickerNest V5 is a **spatial operating system** composed of three coordinated layers. This is the conceptual model — the mental map. The build order (Layer 0 through Layer 6) is the implementation sequence, not the architecture.

Layer A — Spatial Layer

The canvas, entity hierarchy, rendering, z-order, input routing. Everything visual and spatial lives here. This layer does not know about data or logic — it only knows about what exists in space and where.

Layer B — Data Layer

Persistent DataSources — docs, tables, notes, folders, files, custom records. Data exists independently of any widget that displays it. Multiple widgets can reference the same DataSource simultaneously.

Layer C — Interaction Layer

Widgets (GUI views over DataSources and the bus), Scripts (headless automation), Pipelines (visual event routing), and the Event Bus (IPC). This layer gives the canvas behavior.

The OS metaphor fully realized:

OS Concept	StickerNest Equivalent
Environment	Canvas
Spatial objects	Entities
Files / Records	DataSources

OS Concept	StickerNest Equivalent
Applications	Widgets
IPC layer	Event Bus
Scripting / Macros	Scripts + Terminal
App Store	Marketplace
Deployable environment	Canvas as SaaS

Canvas

The infinite 2D (or spatial 3D/VR) workspace where everything lives. The canvas is the scene — the surface on which stickers, entities, and widgets are placed and arranged. Think of it like an isometric diorama: you compose a scene by placing pieces. The canvas handles rendering, zoom, pan, z-ordering, and input routing. It does not know about widget logic — it only knows about what is placed on it and where.

DataSource

A DataSource is a persistent, addressable record that exists independently of any widget instance. DataSources are the **data layer** of StickerNest — the files and records of the OS metaphor.

The key principle: **data is decoupled from display.** A widget does not own its data. It references a DataSource via `dataSourceId`. Multiple widgets can reference the same DataSource simultaneously, and they all stay in sync because they share the same underlying record.

DataSource types:

- `doc` — a rich text document
- `table` — a structured data table (rows + columns with schema)
- `note` — a lightweight plain text or markdown note
- `folder` — a container of other DataSources (like a file system directory)
- `file` — a binary asset (image, audio, video, PDF, etc.)
- `custom` — user-defined schema, for advanced use cases and integrations

DataSource properties:

- `id` — unique identifier
- `type` — one of the types above
- `ownerId` — the user who created it
- `scope` — `canvas | user | shared | public`
- `canvasId` — if canvas-scoped, which canvas it belongs to
- `schema` — optional, for table and custom types
- `metadata` — name, description, tags, etc.
- `createdAt` / `updatedAt`

Scope explained:

- `canvas` — DataSource belongs to a specific canvas, deleted with it
- `user` — DataSource belongs to the user, accessible across all their canvases
- `shared` — accessible to specific collaborators (permission-controlled)
- `public` — readable by anyone with the link

Permissions: DataSource permissions are independent from Canvas roles. A user with Editor role on a canvas does not automatically have write access to all DataSources on that canvas — DataSource ACLs are set separately. Widgets cannot elevate data permissions beyond what the DataSource allows.

Relationship to widgets: A widget is a GUI view over a DataSource. A Doc widget renders a `doc` DataSource. A Table widget renders a `table` DataSource. The widget handles display and interaction; the DataSource holds the actual data. If the widget is deleted, the DataSource persists. If the DataSource is deleted, widgets referencing it show an empty/disconnected state.

Relationship to external integrations: An external database (Notion, Google Sheets, Airtable) maps to a DataSource of type `table` or `custom`. The integration layer syncs the external source into a StickerNest DataSource. The widget always reads from the local DataSource — it never calls the external API directly.

Sticker

The fundamental visual unit of StickerNest. A sticker is an image, GIF, or video asset placed on the canvas — like a scene piece in a diorama. Stickers are spatial and decorative first. They are not programs themselves.

Stickers act as programmable buttons. A sticker never contains logic — it triggers logic. When interacted with, a sticker can perform exactly one of five actions:

1. **Launch a widget** — opens a widget by `widgetId`. The sticker acts as a portal or launcher.
2. **Run a script** — executes a script by `scriptId`. Lightweight automation without a full widget.
3. **Open an external URL** — navigates to or opens a URL in a panel or new tab.
4. **Navigate to another canvas** — transitions the user to a different canvas by `canvasId` or slug.
5. **Emit a custom bus event** — fires a named event directly to the event bus, triggering any subscribed widgets or pipelines.

A sticker never contains logic. It triggers logic.

> **Note on the V4 tagline:** "Every sticker is a computer program" was the original StickerNest philosophy statement. In V5, this has evolved. Stickers are visual assets that can *optionally* become interactive. The more accurate V5 framing is: **every canvas is a living system** — the platform itself is the program, and stickers, widgets, scripts, and pipelines are all ways of giving it behavior.

Properties:

- Position, size, z-index on the canvas
- Asset reference (image, GIF, video URL)
- `action?` — one of: `{ type: 'widget', widgetId }` | `{ type: 'script', scriptId }` | `{ type: 'url', url }` | `{ type: 'canvas', canvasId }` | `{ type: 'event', eventName, payload? }`
- Display label (optional)

Entity

Entities are the canvas objects that live in the engine hierarchy — the building blocks the system actually tracks and manipulates spatially. Every object that has a **position on the canvas** is an entity. Entities live in Layer A (Spatial). They do not have databases. They save to the canvas automatically.

The rule: if it has a position on the canvas, it is a spatial entity.

Entity Types:

- **Text** — a vector text object on the canvas. Has position, size, font, color. Editable by text widgets. Saves to the canvas. Not a database — just spatial text, like text in Figma or Illustrator.
- **Image** — a static image asset placed spatially on the canvas. Has position and size.
- **Vector** — a scalable vector graphic placed on the canvas. Can be a path, shape, icon, or illustration. Editable by vector editing widgets. Supports fill, stroke, path manipulation. SVG-compatible.
- **Shape** — geometric primitives (rect, circle, polygon, line, etc.) placed on the canvas. Distinct from Vector in that shapes are parametric (defined by properties like radius, width) rather than arbitrary paths.
- **Audio** — a sound source placed spatially on the canvas. Has position (relevant in 3D/VR for spatial audio), playback state, volume, loop settings. Editable and triggerable by audio widgets and scripts.
- **Video** — a video object placed spatially on the canvas. Has playback controls, loop, autoplay settings.
- **3D Object** — a three-dimensional mesh or scene object placed in the canvas. Relevant in Spatial/VR mode (Layer 4B). Has position, rotation, and scale in 3D space. Supported formats: GLTF, GLB, OBJ. Editable by 3D widgets.
- **Sticker** — a visual asset (image, GIF, video) placed on the canvas. Can optionally open a widget or run a script on interaction.
- **Widget Container** — a mounted, running widget occupying canvas space. Has position and size. Contains a running widget instance.

Entity vs DataSource — the key distinction:

	Entity	DataSource
Has a canvas position	Yes	No
Lives in	Spatial Layer (A)	Data Layer (B)
Has a database	No	Yes
Opened by widgets	No (it is on the canvas)	Yes (widget is a view over it)
Example	Text block on canvas	Doc, Table, File

Media follows the same rule: An image placed on the canvas is an Image entity (spatial, no database). A photo stored as a persistent record that widgets can open, display, and reference across canvases is a File DataSource. The entity is the spatial presence; the DataSource is the stored record.

Entities follow an ECS (Entity Component System) style hierarchy. They have transform data (position, rotation, scale), lifecycle states, and can be parented to other entities.

Hierarchy

The parent-child tree of entities on the canvas. Entities can be nested — a widget container might be a child of a sticker group, or a group of shapes might be parented to a map layer. The hierarchy controls:

- Transform inheritance (child moves with parent)
- Render order
- Event propagation
- Selection grouping

This is the game-engine layer of StickerNest. It is intentionally separate from widget logic.

Widget

A widget is a self-contained interactive program. Widgets run inside sandboxed iframes (for third-party code) or as trusted inline React components (for built-ins). They communicate with the rest of StickerNest exclusively through the Event Bus.

Widgets are the "apps" of StickerNest. A calendar widget, a Pomodoro timer, a weather display, a note editor — these are all widgets.

A widget does not know where it lives on the canvas. It only knows the event bus.

What saves, and who owns it.

There are four separate things that persist when a widget runs, each owned independently:

1. **Canvas / Data Layer** — entities and DataSources the widget created or modified. Owned by the canvas. Persists automatically the moment the change is made, regardless of whether the widget is open or closed. Closing a widget never undoes canvas changes. This is the shared, persistent truth — visible to all users with access.
2. **Widget instance state** — the widget's own internal runtime state per canvas instance. A map creator widget remembers the current map, zoom level, active tool, and selection. A timer widget remembers how much time is left. Saved via `StickerNest.setState()`, owned by the widget instance on that canvas, restored automatically when the widget is reopened.
3. **Widget instance config** — static configuration for that specific widget instance. Which map layer is active, what color palette is selected, what the default zoom is. Set by the user at setup time, remembered permanently, accessible via `StickerNest.getConfig()`. Does not change during normal use.
4. **User state** — per-user preferences and personalization that follow the user across all canvases. A Spotify widget remembers the user's login. A theme widget remembers the user's preferred color scheme. Saved via `StickerNest.setUserState()` / `StickerNest.getUserState()`. Owned by the user, not the canvas. If the same widget appears on ten canvases, the user state is the same on all ten.

The canvas does not control 2, 3, or 4. The widget does not control 1. All four save automatically and independently.

The Photoshop principle (extended): the paint stays on the canvas when you close the brush tool (canvas owns entities). AND when you reopen the brush tool it remembers your last brush size, opacity, and color (widget owns its state and

config). Two separate persistence mechanisms, both automatic, neither dependent on the other.

A widget can create entities, modify entities, and write to DataSources while it is running. Once it does, those changes belong to the canvas and the data layer — not to the widget. Closing a widget does not undo, hide, or remove anything it created or changed. The canvas automatically persists all entity and DataSource changes as they happen, regardless of which widget made them.

This is the Photoshop principle: you use a brush tool to paint, then close the brush panel. The paint stays. The tool never owned the paint.

The canvas is the source of truth. The widget is the tool.

Widgets read from and write to the canvas and DataSource layer. They do not hold canonical state themselves. When a widget closes, nothing is lost — the widget was never holding anything. It was always writing through to the canvas in real time.

Widget properties:

- Manifest (metadata, version, event contract)
- Capability declarations (what entity types it can interact with)
- Event subscriptions and emissions
- Config schema (user-adjustable settings)
- Sandboxed or inline rendering mode

Widget capability types — widgets declare which entity types they can read or modify. A rich text editor widget declares it works with Text entities. An image editor declares it works with Image entities. This is a compatibility contract, not a type on the widget itself.

How widget storage works:

Widgets are not copied per user. The architecture is:

Thing	Where it lives
Widget bundle (code)	`widgets` Supabase Storage bucket, CDN-served, versioned and immutable

Thing	Where it lives
Widget metadata	`widgets` database table (name, author, manifest, price, screenshots)
User's installed widgets	`user_installed_widgets` join table — a pointer, not a copy
Draft / unpublished widgets	Same bucket under user-scoped path, `published: false`
Widget instance config/state	`widget_instances` table, per canvas per user

Installing a widget writes one row to `user_installed_widgets`. The user gets a reference to the shared CDN-served bundle — not their own copy. This means a popular widget is served from one cached file to all users.

Widget Types

Not a formal type system on widgets — rather, a loose categorization for discovery and organization purposes:

- **Utility** — timers, calculators, counters, clocks
- **Content** — notes, text editors, image viewers, markdown
- **Data** — charts, spreadsheets, weather, stocks, feeds
- **Social** — chat, presence, reactions, shared cursors
- **Integration** — Google Calendar, Notion, GitHub, Spotify, etc.
- **Creative** — drawing tools, audio players, video players
- **Game / Interactive** — mini-games, simulations, interactive maps
- **Spatial / 3D** — Three.js widgets for the spatial canvas

These categories exist for browsing and marketplace filtering, not for architecture.

Panel System

The panel system allows users to attach persistent side panels to their canvas workspace. Panels exist outside the canvas itself — they don't have a position on

the canvas and are not entities. A panel is a UI surface anchored to the edge of the workspace.

When adding a panel, users choose one of three panel types:

- **Docker Panel** — a container for widgets (see Docker below)
- **Web Preview Panel** — renders an external webpage inside the panel
- **Canvas Preview Panel** — renders another canvas inside the panel (live, interactive)

Multiple panels can exist simultaneously. Panels can be resized, collapsed, and repositioned to different edges of the workspace.

Docker

A Docker is both a panel type and a widget type. Understanding both:

As a panel: The Docker panel is a system-level parent container that users add to their workspace to organize widgets. Instead of widgets floating freely on the canvas, users drag them into the Docker where they become its children. The Docker panel can display its children in two layouts:

- **Tabbed** — each child widget gets its own named tab
- **Scrollable** — all child widgets stacked in a single scrollable column

As a widget type: Docker is a first-class widget that any user can build in Widget Lab. A custom Docker widget follows the same `supportsChildren: true` contract as any parent widget — it just happens to be a container whose purpose is organizing other widgets. This means users can build specialized Dockers: a media control toolbar Docker, a game HUD Docker, a dashboard Docker with a specific layout.

Dockers are saveable and portable. A user builds a Docker (system or custom), populates it with widgets, and saves it as a named configuration. That saved Docker can be opened on any canvas — it remembers its children and their configurations. This makes Dockers reusable toolbars, dashboards, and control panels.

The Docker is the clean answer to widget sprawl. Widgets that would otherwise scatter across the canvas live inside a Docker instead. The canvas stays clean; the Docker holds the tools.

Key rules:

- Every widget inside a Docker is a child of that Docker in the entity hierarchy
- Children communicate through the event bus as normal — the Docker does not mediate communication
- A widget can be moved out of a Docker back onto the canvas at any time
- Dockers can be nested (a Docker inside a Docker) up to 2 levels deep
- The system Docker panel is the official implementation; custom Docker widgets are user-built variants

---

Widget Lab

The in-app development environment for creating and editing widgets. Widget Lab is where you build, test, and publish widgets without leaving StickerNest. It provides:

- A code editor (HTML/JS/CSS, single-file widget format)
- Live preview in 2D, 3D, and isolated contexts
- Event bus inspector to test event contracts
- A node graph view for composing widget logic visually
- AI-assisted generation (Claude, GPT-4, Gemini)
- A publish pipeline to the Marketplace

Widget Lab is for creators and developers. Regular users never need to open it.

Event Bus

The nervous system of StickerNest. The event bus is a typed pub/sub system that sits at the kernel layer — below widgets, below the canvas, below everything. All

communication between widgets happens through the bus. Widgets never talk to each other directly.

How it works:

- Widgets emit typed events (`timer.complete`, `note.updated`, `track.changed`)
- Widgets subscribe to event types they care about
- The bus routes events to all subscribers
- Events can be wired explicitly (via Pipelines) or caught by any subscriber

The event bus is the reason StickerNest is composable. Any widget can react to any event from any other widget, as long as the types match.

VR / Spatial events:

All bus events carry an optional `spatial` field for 3D context. This field is `undefined` for standard 2D canvas events and populated for events that originate from VR interactions or spatial entity actions.

```
spatial?: {
  position: Vector3      // 3D world position of the interact
ion origin
  rotation: Quaternion   // orientation at time of event (e.
g. hand direction)
  normal: Vector3        // surface normal if the event origi
nated on a mesh
}
```

This is fully backward compatible — existing widgets that don't check for `spatial` continue to work unchanged. Widgets that want to be spatially-aware can read `spatial` and respond to 3D position data.

Pipelines

A Pipeline is a visual, multi-step connection between two or more widgets via the event bus. It is the no-code wiring layer — users connect widget outputs to widget inputs by drawing lines between them in the connection graph.

A pipeline is a chain of nodes. Each node is one of:

- **Widget node** — a running widget instance (as source or destination)
- **Script node** — an optional transform or logic step in the middle of the chain
- **Filter node** — passes events through only when a condition is met
- **Transform node** — remaps event payload fields from one schema to another

Examples:

- Pomodoro timer → Spotify widget ("when timer completes, play reward track")
- Weather widget → Script node ("if temp > 80") → Background color widget
- Task widget → Notion database widget ("when task checked, log to Notion")

Pipelines are visible and editable in the connection graph overlay in the Canvas Editor. A script node in a pipeline is the same scripting system described under Widget Scripting — pipelines and scripts are complementary, not competing. The pipeline is the visual wire; the script is optionally one of the nodes on that wire.

Canvas Gallery

The top-level view for browsing, organizing, and launching your canvases (workspaces). Each canvas is a saved scene — a named collection of stickers, entities, and widget configurations. The Canvas Gallery is your home screen. From here you can create a new canvas, open an existing one, duplicate, share, or delete.

Map Layer

A specialized canvas layer designed for geographic or spatial map content. The map layer renders a real or fictional map (using Leaflet, Mapbox, or a custom tile system) as a canvas background, and allows entities and stickers to be pinned to geographic coordinates. Used for:

- Game world maps with interactive stickers
- Real-world dashboards (weather by location, news by region)
- Travel planning canvases

- Collaborative storytelling maps

The map layer is just another layer in the canvas hierarchy — it is not a separate mode.

Social Widgets

Widgets designed for shared, multi-user interaction. Social widgets are aware of other users on the same canvas and enable real-time collaboration features:

- **Presence indicators** — see where others are on the canvas
- **Shared cursors** — see others' cursors in real time
- **Reactions** — emoji reactions tied to canvas positions
- **Chat** — canvas-contextual chat threads
- **Voting / polling** — collaborative decision widgets

Social widgets use the event bus like all other widgets, but their events are broadcast to all users on the canvas via the real-time sync layer (Supabase Realtime).

Marketplace

The store where widgets are discovered, shared, and sold. The Marketplace is where:

- Users browse and install widgets built by the community or by Anthropic/StickerNest
- Widget creators publish their work (via Widget Lab's publish pipeline)
- Premium widgets are purchased or subscribed to
- Collections and bundles are curated

The Marketplace is a Layer 5 concern — it sits above the core platform. The architecture should support it from day one, but it is not required for the initial build.

Multiuser Canvas

A canvas that multiple users can view and interact with simultaneously in real time. Multiuser canvases are the social foundation of StickerNest — every canvas is potentially multiuser, but presence and editing rights are controlled by the owner. Modes:

- **View-only** — others see the canvas but cannot edit
- **Collaborative edit** — invited users can move, add, and configure stickers
- **Live session** — all users see each other's cursors, selections, and changes instantly via Supabase Realtime

Multiuser state (cursor positions, presence, live changes) flows through the Event Bus and is synced via the real-time layer. The canvas owner controls permissions. Social widgets surface this hidden layer visually.

Conflict Resolution Model

StickerNest uses different sync strategies depending on the type of data being edited concurrently. These are not competing systems — each is matched to the appropriate data type.

Canvas entities (positions, transforms, properties) — Last-write-wins. Spatial changes are low-stakes and fast-moving. The social layer shows presence cursors so users can see each other working and naturally avoid collisions. Silent resolution is appropriate here.

Doc DataSources — Yjs CRDT. Rich text co-editing requires operational transforms so no keystrokes are ever lost. Both users' changes merge intelligently in real time. Same model as Google Docs.

Table and Custom DataSources — Revision-based conflict detection with LWW fallback. Each row carries a `revision` counter. If two users submit writes to the same row concurrently, the write with the lower revision is rejected and the client must re-fetch and retry. If the platform cannot determine revision order (e.g. near-simultaneous writes within clock skew), LWW applies. This supersedes the earlier "conflict indicator" model.

Note DataSources — Last-write-wins. Notes are lightweight enough that full CRDT is overkill.

Data type	Strategy	Why
Canvas entities	Last-write-wins	Spatial, fast, presence prevents most collisions
Doc DataSource	Yjs CRDT	Rich text needs operational transforms
Table/Custom DataSource	Revision-based, LWW fallback	Revision counter rejects stale writes; LWW on clock skew
Note DataSource	Last-write-wins	Lightweight, low collision risk

---

Canvas Slug

A human-readable URL identifier for a canvas. Instead of `/canvas/3f8a92bc`, a canvas with a slug is accessible at `/canvas/my-game-world` or `stickernest.app/u/kimber/morning-dashboard`. Slugs are:

- Unique per user (or globally for public canvases)
- Set by the user or auto-generated from the canvas name
- Used for sharing, embedding, and deep-linking
- Required for public and SaaS canvases

Canvas as SaaS

The model where a StickerNest canvas becomes a deployable product in its own right. A creator builds a canvas — a dashboard, a game, a tool — and publishes it with a slug as a standalone web app that others pay to use or subscribe to. The canvas owner earns revenue; StickerNest takes a platform fee.

This is the long-term monetization unlock: StickerNest is not just a tool you use, it is a platform you build on and sell from. Every canvas is a potential SaaS product.

Canvas Editor

The primary editing interface for a canvas. The canvas editor is what users see when they open a canvas in edit mode. It provides:

- Sticker and entity placement, moving, resizing, z-ordering
- Widget attachment and configuration
- Layer management (including map layer)
- Pipeline wiring via the connection graph
- Mode switching (2D, spatial/3D, VR)
- Undo/redo, snap-to-grid, alignment tools
- Keyboard shortcuts and controller/hand input support

The canvas editor is distinct from Widget Lab (which is for building widgets) and the Canvas Gallery (which is for managing canvases).

StickerNest as Game Engine Protocol

At its core, StickerNest is a protocol — a defined way for visual entities, interactive widgets, and event-driven systems to coexist and communicate. Like a game engine, it provides:

- A scene graph (entity hierarchy)
- An event system (the bus)
- A runtime (widget sandboxes)
- Input handling (pointer, keyboard, controller, hand tracking)
- Rendering modes (2D canvas, Three.js spatial, WebXR)
- A scripting layer (widgets as programs)

The protocol is open: anyone can implement it. The official StickerNest platform is the reference implementation. Third parties can build their own runtimes against the same protocol.

StickerNest as Operating System

Zooming out further: StickerNest is an OS for interactive visual spaces. The canvas is the desktop. Stickers are icons and windows. Widgets are apps. The event bus is the IPC layer. The marketplace is the app store. The Widget Lab is the IDE. Canvases as SaaS are deployable environments.

This framing guides long-term decisions: just as an OS provides stable primitives that apps are built on top of, StickerNest provides stable primitives (bus, entities, runtime) that widgets and canvases are built on top of.

Social Layer

The hidden infrastructure that powers all real-time, multi-user, and social features. The social layer sits between the event bus and the network — it serializes events and state changes and broadcasts them to all connected users on a canvas. It is not a widget; it is platform infrastructure. Social widgets (chat, cursors, reactions, presence) are the visible surface of this layer. The social layer handles:

- User presence (who is on the canvas right now)
- Conflict resolution for simultaneous edits
- Event broadcasting via Supabase Realtime
- Permission enforcement per canvas and per entity

Quest 3 / VR Mode

StickerNest runs natively in VR via WebXR on the Meta Quest 3 (and compatible headsets). In VR mode:

- The canvas becomes a spatial environment you stand inside
- Hand tracking maps pinch, grab, and point gestures to canvas interactions
- Controller input (thumbstick, trigger, grip) maps to navigation and editing
- Stickers and entities exist in 3D space with depth and scale
- Widgets render as floating panels or volumetric objects
- The event bus works identically — widgets don't know or care if they're in VR

VR mode is a rendering target, not a separate platform. The same canvas, the same widgets, the same event bus — just a different renderer and input system.

Widget Skins / Themable Tokens

Widget skins are visual style overrides applied to a widget instance without changing its logic. Every widget exposes a set of **design tokens** — CSS custom properties like `--sn-bg`, `--sn-accent`, `--sn-font-family`, `--sn-border-radius`, `--sn-text-color` — that the host platform can set from outside the iframe sandbox.

A skin is a named collection of token values. Users can apply a skin to a single widget instance, or to all widgets on a canvas at once. Creators can ship widgets with multiple built-in skins (e.g. "Dark", "Light", "Neon", "Minimal") or expose all tokens for full customization.

How it works:

- Widget declares its supported tokens in its manifest under `themeContract`
- Host injects token values into the iframe via postMessage on load and on theme change
- Widget applies tokens to its root element as CSS custom properties
- Widgets that don't declare tokens still receive the global StickerNest theme tokens as a baseline

Skins can be shared, sold in the Marketplace, and applied as part of a canvas theme.

StickerNest UI Theming

The StickerNest platform shell (Canvas Gallery, Canvas Editor, Widget Docker, Widget Lab, Marketplace) is itself fully themable. The UI theming system uses a global design token set that controls the look of every surface in the app.

Token categories:

- `--sn-color-*` — brand, surface, border, text, error, success colors
- `--sn-radius-*` — border radius scale

- `--sn-spacing-*` — spacing scale
- `--sn-font-*` — font family, size scale, weight
- `--sn-shadow-*` — elevation levels
- `--sn-motion-*` — animation duration and easing

Built-in themes: Dark (default), Light, High Contrast, Midnight, Sand

Custom themes: Users can create and save custom themes. Creators can ship a canvas with a locked theme so the whole experience feels branded. Enterprise users can white-label the entire UI.

The widget token system and the UI theming system share the same token naming convention so widgets naturally inherit the canvas's look when using standard tokens.

The Interaction Stack

StickerNest has four layers of interaction with the event bus, each serving a different user type. They share the same underlying infrastructure and do not compete:

Layer	Who it's for	How it works
Pipelines	Everyone	Visual no-code wiring between widgets
AI Command Bar	Everyone	Natural language → bus events
Event Bus Terminal	Power users / developers	Direct CLI → bus events
Scripts	Power users / developers	Persistent logic that lives on the bus

Each layer is a different face on the same system. None can do anything the others categorically cannot — they just have different interfaces and levels of expressiveness.

AI Command Bar

A natural language input layer that sits above the event bus. The command bar lets users control StickerNest and their widgets through plain language instead of UI interactions. It acts as an AI translation layer — the user types or speaks a

command, the AI interprets intent, and emits the correct bus event or performs the matching UI action.

Examples:

- "Set all timers to 25 minutes" → emits `timer.set` to all timer widgets
- "Play my focus playlist" → emits `spotify.play` with the matching playlist
- "Move the calendar sticker to the top right" → updates the sticker's transform
- "Create a sticky note that says ship it" → creates a Text entity
- "Create a script that logs every event to my Notion widget" → generates and saves a script

Under the hood, the AI Command Bar generates Event Bus Terminal commands and executes them. The two features share infrastructure — the command bar is the friendly face of the terminal.

The command bar is not a separate system — it is a natural language frontend to the same event bus and canvas actions that already exist.

Event Bus Terminal

A developer-facing CLI interface to the event bus. The terminal lets power users and developers interact with the bus directly, without going through the GUI or AI translation. It is a debugging and authoring tool, not a runtime system.

What it can do:

- `emit timer.start {"duration": 25}` — emit any event directly to the bus
- `subscribe weather.*` — watch a live stream of matching events
- `list widgets` — see all registered widget instances and their event contracts
- `list scripts` — see all scripts attached to this canvas
- `run script my-script` — execute a named script manually
- `pipeline list` — see all active pipelines
- `history 50` — show last 50 events from the bus ring buffer

What it cannot do:

- Access the file system
- Execute arbitrary Node.js or shell commands
- Bypass the widget sandbox or RLS policies
- Do anything a sandboxed widget could not already do

Security model: The terminal is scoped to the event bus and canvas actions only. It runs inside the same permission model as widgets — it cannot reach outside the StickerNest sandbox. A user without edit permission on a canvas cannot use the terminal to mutate it.

Where it lives: The terminal is a panel type (alongside Docker, Web Preview, Canvas Preview). Users add a Terminal panel to their workspace. It is also accessible as a floating overlay via a keyboard shortcut (` backtick, like most game engines).

Relationship to other layers: The AI Command Bar generates and executes terminal commands under the hood. Scripts can be authored as one-liners directly in the terminal and saved. The terminal is the lowest-level interface a user ever needs — anything below it is platform internals.

Mobile / Touch Mode

StickerNest is fully functional on phones and tablets. Mobile is a first-class target, not a degraded experience. The interaction model adapts to touch:

- **Tap** = select / activate (replaces click)
- **Tap + hold** = context menu (replaces right-click)
- **Drag** = move entity or pan canvas
- **Two-finger pinch** = zoom canvas
- **Two-finger rotate** = rotate selected entity
- **Bottom sheet** = replaces sidebars and panels (slides up from bottom)
- **Widget docker** = collapses to a floating action button on mobile

The canvas editor has a mobile layout. Widget Lab is desktop-only. The Canvas Gallery and view-mode canvases are fully mobile-optimized. Widgets are

responsible for being responsive within their container — the Widget SDK provides a `StickerNest.onResize(handler)` callback.

Collaboration Roles

Every canvas has per-user permission roles. Roles control what a collaborator can do:

- **Owner** — full control, billing, delete, share settings
- **Editor** — can add/move/configure entities and widgets, cannot change sharing settings
- **Commenter** — can leave annotations and comments, cannot edit entities
- **Viewer** — read-only access, can see the canvas and interact with widgets in view mode

Roles are set per-canvas per-user. The same user can be an Editor on one canvas and a Viewer on another.

Canvas Version History

StickerNest automatically saves snapshots of a canvas at regular intervals and on significant actions. Version history lets users:

- Browse a timeline of canvas states
- Preview any past version
- Restore a previous version (replaces current state)
- Create named versions ("Before the big redesign", "v1.0 launch")

Version history is stored in Supabase as canvas state diffs. Retention depends on plan tier.

Annotations / Comments

Spatial comments pinned to specific positions or entities on a canvas. Annotations are distinct from chat — they are persistent, addressable, and tied to a location rather than a timeline.

- Anyone with Commenter role or above can leave an annotation
- Annotations appear as pins on the canvas, expanding on click
- Annotations can be resolved (hidden but preserved)
- Useful for design review, async collaboration, feedback on shared canvases

---

Notifications

In-app and email alerts for events that matter to the user:

- Collaborator joined your canvas
- Someone left an annotation on your canvas
- A widget you're subscribed to published an update
- A marketplace widget you purchased has a new version
- Someone purchased your published widget or canvas
- Canvas version auto-saved
- Realtime session invite

Notification preferences are configurable per category. Delivery channels: in-app bell, email digest.

Subscription Tiers

StickerNest is a SaaS product with tiered plans that gate features and usage limits:

- **Free** — limited canvases, basic widgets, no realtime collaboration, no slug, no Marketplace publishing
- **Pro** — unlimited canvases, full widget library, realtime collaboration (up to 5 seats), canvas slugs, basic analytics
- **Creator** — everything in Pro + Widget Lab access, Marketplace publishing, revenue sharing, version history
- **Enterprise** — everything in Creator + white-label, SSO, SLA, unlimited seats, custom integrations, dedicated support

The Canvas as SaaS model is a Creator+ feature. VR mode is Pro+.

Offline / Local-First

StickerNest works without a network connection for single-user canvas editing. The local-first model means:

- Canvas state is kept in local storage / IndexedDB as the primary working copy
- Changes sync to Supabase when connection is available
- Offline changes are queued and replayed on reconnect
- Conflicts between offline edits and server state are resolved with last-write-wins (with user prompt on major conflicts)
- Realtime collaboration and social features require connection — they gracefully degrade when offline

---

Widget Versioning

Widgets follow semantic versioning (semver). The manifest declares a version string. The platform tracks which version of a widget each canvas instance is pinned to.

- **Auto-update** — canvas always runs the latest version of installed widgets (default for free widgets)
- **Pinned** — canvas is locked to a specific version (available for paid/creator tier)
- **Breaking changes** — major version bumps require explicit user upgrade
- **Backwards compatibility** — minor and patch updates must not break existing event contracts
- Widget Lab enforces this by warning creators when a manifest change would be a breaking change

---

Accessibility (a11y)

StickerNest is built to WCAG 2.1 AA standard. Key commitments:

- Full keyboard navigation of the canvas editor (select, move, resize, configure without a pointer)
- Screen reader support for the Canvas Gallery, settings, and all modal/panel UI
- Focus management in widgets — widgets must not trap focus unexpectedly
- Sufficient color contrast in all built-in themes (High Contrast theme for users who need it)
- Reduced motion mode respects `prefers-reduced-motion`
- Touch targets minimum 44×44px on mobile

The Widget SDK provides a11y guidance and validators. Marketplace submissions are reviewed for basic a11y compliance.

Internationalization (i18n)

StickerNest is architected for multi-language support from day one, even if the initial launch is English-only.

- All UI strings are externalized into translation files (no hardcoded copy in components)
- RTL layout support via CSS logical properties
- Date, number, and currency formatting via `Intl` APIs
- Widget manifests include a `locale` field for localized names and descriptions
- Translation contributions accepted via open-source workflow post-launch

---

Cross-Canvas Widget Communication

Widgets on different canvases can communicate through the event bus via named channels. This works for both single-user (two canvases open in different tabs) and multiuser (two users each on their own canvas that are linked).

Single-user cross-canvas — a user has Canvas A open in one tab and Canvas B in another. A widget on A can emit to a shared named channel; a widget on B subscribed to that channel receives it. The channel is brokered server-side via Supabase Realtime.

Multiuser cross-canvas — two users on separate canvases that have been explicitly linked. Useful for shared dashboards, game master / player setups, or broadcast scenarios. One canvas acts as a source, another as a subscriber.

How it works:

- Widgets declare cross-canvas channels in their manifest under `crossCanvasChannels`
- Canvas owner explicitly links two canvases and maps channels between them
- The event bus routes cross-canvas events through the social layer's Realtime broker
- Permissions: cross-canvas communication requires both canvas owners to approve the link
- Latency target: cross-canvas events delivered within 500ms

Widget State Persistence

Widget state is saved automatically across sessions. When a user closes and reopens a canvas, every widget instance resumes exactly where it left off.

Two levels of persistence:

- **Instance state** — the runtime state of a specific widget on a specific canvas (e.g. a timer widget's current countdown, a note widget's text content). Stored in Supabase per sticker/widget-container instance.
- **User state** — state that follows the user across all canvases (e.g. a user's Spotify login inside the Spotify widget, their Notion connection). Stored per user, not per canvas instance.

Widgets access persistence via the SDK: `StickerNest.setState()` / `StickerNest.getState()` for instance state, and `StickerNest.setUserState()` / `StickerNest.getUserState()` for cross-canvas user state. The host handles all storage — widgets never touch localStorage or IndexedDB directly.

Database Widget Integration

Widgets can connect to external databases and data sources — Notion, Airtable, Google Sheets, Supabase, REST APIs, and more. This is handled through the **Integration layer**: a set of authenticated connectors that widgets can request access to via the SDK.

How it works:

- User connects an integration in Settings (OAuth or API key)
- Widget manifest declares which integrations it needs under `integrations`
- Widget SDK exposes `StickerNest.integration(name).query(params)` — a sandboxed data access interface
- The host proxies the request through the integration layer (widget never holds credentials directly)
- Results are returned as typed data the widget can render

Supported integrations (planned): Notion databases, Google Sheets, Airtable, Supabase tables, generic REST/GraphQL endpoints, local CSV/JSON upload.

This makes database widgets possible: a widget that renders a Notion database as a kanban board, a widget that shows live rows from a Supabase table, a spreadsheet widget backed by Google Sheets.

Database Manager

A first-class platform page, accessible from the main navigation alongside Canvas Gallery and Settings. The Database Manager is where users see, create, and manage their entire data layer — both native StickerNest DataSources and external connections.

The Database Manager has two sections:

My Databases

All DataSources the user owns or has access to. Filterable by type (doc, table, note, folder, file, custom) and scope (canvas, user, shared, public). Users can:

- Create new DataSources directly — not through a widget, not through a canvas

- Define schema for table and custom types (column names, field types, required fields)
- Set scope and permissions (who can read, write, or comment)
- See which canvases and widgets are currently referencing each DataSource
- Preview DataSource content inline
- Rename, duplicate, delete, or change scope of any DataSource they own
- Share a DataSource directly with another user (independent of canvas sharing)

Connections

All external integrations in one place. Users authenticate once per service. Each connection shows:

- Sync status (connected, syncing, error, disconnected)
- Last synced time
- Which DataSources it maps to
- Manual re-sync button
- Re-auth flow when OAuth tokens expire

Supported connections: Notion, Airtable, Google Sheets, Supabase, REST/GraphQL endpoints. Credentials are stored encrypted in StickerNest — never exposed to widgets.

Key principle: DataSources created in the Database Manager are fully independent of any canvas or widget. They exist in the user's data layer and can be connected to any widget on any canvas at any time. A user might create a `table` DataSource for their project tasks in the Database Manager, then use it simultaneously in a kanban widget on their work canvas and a list widget on their mobile canvas.

Storage Buckets

StickerNest uses Supabase Storage organized into typed buckets. Buckets hold the raw bytes. File DataSources hold the addressable, permission-controlled

references to those bytes. The two work together: the bucket is the infrastructure, the File DataSource is the named record a user or widget interacts with.

Bucket structure:

Bucket	Contents	CDN-served?
`media/images`	Uploaded images (PNG, JPG, WebP, SVG)	Yes
`media/gifs`	Uploaded and optimized GIFs	Yes
`media/video`	Uploaded video files (MP4, WebM)	Yes
`media/audio`	Uploaded audio files (MP3, WAV, OGG)	Yes
`media/3d`	3D model files (GLTF, GLB, OBJ)	Yes
`stickers`	Sticker asset packs (images, GIFs)	Yes
`widgets`	Widget code bundles (versioned, immutable)	Yes
`canvas-thumbnails`	Auto-generated canvas preview images	Yes
`user-avatars`	User profile photos	Yes
`exports`	User-generated exports (JSON, PDF, PNG)	No (private)

Key principles:

- An asset uploaded once lives in a bucket and can be referenced by any canvas, any sticker, any entity, any widget — across all of a user's canvases. No re-uploading.
- Widget bundles are stored at a versioned, immutable path: `widgets/{widgetId}/{version}/index.html`. A published version never changes. A new version creates a new path.
- Bucket files are not directly accessible to widget iframes. Widgets access media through the SDK, which proxies through the host. The bucket URL is never exposed to untrusted widget code.
- All user-uploaded assets are scoped to the uploading user via RLS. Public assets (published sticker packs, published widgets) are readable by all.
- Asset size limits enforced at upload: images 5MB, GIFs 15MB, video 50MB, audio 20MB, 3D models 50MB.

Relationship to File DataSource: When a user uploads a media file, two things happen simultaneously: the file is written to the appropriate bucket, and a `file`

DataSource record is created pointing to it. The DataSource is what the user sees in their Library and what widgets reference. The bucket path is an implementation detail.

The Library

A first-class platform page, accessible from the main navigation. The Library is the user's unified creative asset browser — a single place to see and manage everything they own, have built, or have installed, across all canvases.

The Library does not store anything new. It is a unified view over assets that already exist in buckets, the widget registry, the DataSource system, and saved configurations. It is a browsing and organization layer.

Library sections:

Media — all uploaded or generated assets (images, GIFs, video, audio, SVGs, 3D models). Organized by type, searchable by name. Drag any asset directly onto a canvas from the Library. Assets uploaded to any canvas automatically appear here.

Stickers — sticker collections the user owns, has downloaded, or has built. Includes user-uploaded sticker packs and Marketplace packs they've installed. Browseable by collection, searchable by name.

Widgets — all widgets the user has installed, built, or drafted. Subsections: Installed (from Marketplace), Mine (built in Widget Lab, published or draft), Built-ins (system widgets). Each widget shows version, install count (if published), and a launch/edit button.

Pipelines — saved pipeline templates the user has built. Reusable across canvases. Shows the node chain, the widgets involved, and a one-click "apply to canvas" action.

Scripts — saved scripts. Searchable, taggable, with a preview of the script code and a one-click attach action.

Dockers — saved Docker configurations (Docker + its children). Openable on any canvas.

Templates — canvas templates the user has saved or purchased. Includes their own saved canvases marked as templates.

Key principle: The Library is the memory of the platform. Anything reusable lives here. A user never has to upload the same image twice, rebuild the same Docker, or recreate the same pipeline. Everything made once is available everywhere.

Widget Scripting

An optional scripting layer that allows advanced users to write small programs that run on the event bus without building a full widget. Scripts are lightweight — a few lines of JavaScript that subscribe to events, transform data, and emit new events.

Scripts are the "power user automation" layer, like keyboard macros for the event bus. They live in a sandboxed runtime (same iframe security model as widgets) and have access to the Widget SDK.

Use cases:

- "When my timer completes AND it's after 6pm, emit a 'day-done' event"
- "Every time any widget emits a data event, log it to my Notion database widget"
- "Transform the event payload from Widget A's format to Widget B's format"

Scripts are authored in Widget Lab (a lightweight script mode, not the full IDE), or via the AI command bar ("create a script that..."). Scripts are not publishable to the Marketplace as standalone items but can be bundled with a widget or canvas template.

Desktop Mode

> ⚠️ **Architectural rule:** Desktop Mode is a **rendering strategy**, not a separate canvas type. No new entity types exist for Desktop Mode. The same nine entity types render differently — that is all.

An optional canvas view that transforms the canvas from a spatial scene into a functional desktop environment. In Desktop Mode:

- The canvas background becomes a desktop wallpaper
- **StickerEntity** renders as a desktop icon — double-click to launch its associated widget or run its action

- **WidgetContainerEntity** renders as a resizable, draggable window (like an OS window) — minimizable, maximizable, snappable
- **System Docker** renders as a bottom dock (macOS-style) or taskbar
- A taskbar appears showing open windows

Desktop Mode and Scene Mode (the default spatial view) are two views of the **same canvas**. The underlying entity hierarchy, event bus, widget system, and all persistence use the same tables and stores as Scene Mode. Mode is purely a rendering and interaction preference — not a different data model, not a different entity set.

This makes StickerNest a literal operating system metaphor made real: the canvas is the desktop, stickers are icons, widgets are apps, the docker is the dock, and the event bus is the OS IPC layer.

Parent/Child Widget Nesting

Widgets (and widget containers) can be nested inside other widgets. A parent widget hosts child widgets within its own layout, managing their size and position. This is distinct from the entity hierarchy (which is about canvas-level parenting) — widget nesting is about UI composition within a single widget container.

Use cases:

- A **Dashboard widget** that hosts a clock, a weather widget, and a task list as children in a grid layout
- A **Folder widget** that contains doc widgets, image widgets, and note widgets
- A **Tab widget** that renders different child widgets per tab
- A **Splitview widget** that shows two child widgets side by side

How it works:

- Parent widget declares `supportsChildren: true` in its manifest
- Children are registered widget instances, each with their own SDK context and state
- Parent controls layout (size, position of child frames) but not child content

- Events flow normally — children communicate through the bus, not through the parent
- Children can be dragged out of a parent to become standalone instances

---

Doc Widget

A first-class built-in widget that provides a rich text document experience directly on the canvas. The Doc widget is StickerNest's equivalent of a Notion page or Google Doc — a full editing surface with headings, paragraphs, lists, inline images, code blocks, and embeds.

Doc widgets can be:

- Standalone (a note or document on a canvas)
- Nested inside a Folder widget or Dashboard widget
- Linked to an external database (backed by a Notion page or Google Doc via integration)
- Shared and collaboratively edited via the social layer (same real-time model as the canvas)

Folder Widget

A container widget that organizes other widgets and assets into a named, openable folder. In Desktop Mode, a Folder widget looks and behaves like a desktop folder — double-click to open, shows contents as a grid or list. In Scene Mode, a Folder widget appears as a sticker that expands into a panel when clicked.

> ⚠️ **Persistence correction:** Folder contents are stored in a **Folder DataSource**, not in widget instance state. Widget instance state for a Folder widget stores only UI/view preferences (current view mode, sort order, scroll position). Folder items persist across sessions via the DataSource and are visible to all users with access to that DataSource.

Folder items may reference: DataSources (doc, table, note, file, custom), widgets, canvases, external URLs, or uploaded files. Each item is an addressable record in

the Folder DataSource — not a copy, a reference.

DataSource multi-view rule: A Folder DataSource (like all DataSources) can be referenced by multiple widgets simultaneously. Two Folder widgets on different canvases can reference the same Folder DataSource — both show the same contents, and additions in one are immediately visible in the other. This applies equally to Doc, File, and Note DataSources: multiple widgets referencing the same DataSource all sync to the same underlying record.

Canvas View Modes

A canvas can be rendered in one of two primary view modes. The user switches between them per-canvas, and the preference is saved.

Scene Mode (default) — the spatial, isometric-inspired layout. Stickers and entities are placed freely across an infinite canvas. This is the StickerNest V4-style experience: compose a scene, place stickers, wire widgets together.

Desktop Mode — the canvas becomes a managed desktop environment. Widget containers become windows. Stickers become icons. A taskbar and dock appear. This is the OS metaphor made literal.

Both modes share: the same entity hierarchy, the same event bus, the same widget runtime, the same social layer, the same data. Mode is purely a presentation and interaction layer switch.

Future planned modes: **Board Mode** (Kanban-style columns), **Grid Mode** (spreadsheet-style snapping).

Scene → Desktop Mode entity transition rule:

When a canvas switches to Desktop Mode, every existing widget container automatically becomes a window. The entity's current canvas size becomes the initial window size. The entity's current canvas position becomes the initial window position. No data is lost. No user prompt is required. Switching back to Scene Mode restores the original spatial layout.

Canvas Embedding

Any public canvas can be embedded in an external website via iframe. Embedding is a view-mode-only experience — no canvas editor chrome, no toolbar, just the canvas surface and its widgets.

How it works:

- Canvas owner generates an embed code snippet from canvas share settings
- Embed URL format: `stickernest.app/u/{username}/{slug}?embed=true`
- Embedded canvas respects the canvas's public/private setting — private canvases cannot be embedded
- Widgets run normally inside an embedded canvas — all interactivity works
- Canvas editing, annotations, and social features are disabled in embed mode
- Embed can be restricted to specific domains (allowlist in canvas settings)
- Canvas as SaaS: the embed is the primary delivery mechanism for a SaaS canvas product

---

Guest / Anonymous Access

Public canvases are accessible without an account. Guest users can:

- View the canvas in view mode
- Interact with widgets (click, use interactive widgets)
- See other users' presence cursors

Guest users cannot:

- Edit any canvas content
- Leave annotations or chat messages
- Install widgets
- Save canvas state (their widget interactions are session-only)

Guest presence appears as "Guest" with a randomly assigned color. Creating content of any kind (annotations, chat) prompts the guest to create an account.

URL Routing

The full URL structure for StickerNest:

URL pattern	What it is
`stickernest.app`	Marketing / landing page
`stickernest.app/gallery`	Canvas Gallery (authenticated)
`stickernest.app/library`	Library (authenticated)
`stickernest.app/databases`	Database Manager (authenticated)
`stickernest.app/marketplace`	Marketplace (public)
`stickernest.app/settings`	Settings (authenticated)
`stickernest.app/u/{username}/{slug}`	User canvas (public or gated)
`stickernest.app/c/{slug}`	Globally unique canvas slug (Creator+)
`stickernest.app/u/{username}/{slug}?embed=true`	Embedded canvas
`stickernest.app/lab`	Widget Lab (authenticated, Creator+)
Custom domain	Enterprise white-label only

Slug rules:

- Standard slug: unique per user (`/u/{username}/{slug}`)
- Global slug (`/c/{slug}`): unique across all of StickerNest, Creator+ feature
- Slugs are alphanumeric + hyphens only, max 64 characters
- Slug changes are permanent redirects (old slug 301s to new slug for 90 days)

Widget Hot-Reload Policy

What happens when a widget publisher releases an update while users have running instances of that widget on their canvases.

Three modes, applied based on widget type and user tier:

Auto-update (default for free widgets): The update is applied silently the next time the canvas is opened. Mid-session instances are never interrupted. On the next canvas load, the new version is fetched from CDN.

Update-available prompt (default for paid and installed widgets): When a new version is available, an update badge appears on the widget instance in the canvas. The user chooses when to apply it. Canvas keeps running the old version until the user explicitly updates.

Pinned (Creator+ feature): The canvas is locked to a specific version forever. No update badge. No automatic updates. User must manually unpin to update.

Breaking changes (major version bump): Always treated as update-available prompt regardless of tier. A major version change is never applied automatically.

Performance Budgets

Hard targets that must not be regressed. Tested in CI on every build:

- **Canvas FPS** — 60fps for up to 100 entities on mid-range hardware; 30fps minimum
- **Initial load** — Time to interactive under 3 seconds on 4G connection
- **Widget load** — Iframe sandbox initialized and READY within 500ms
- **Event bus latency** — emit-to-handler under 1ms for local events
- **Realtime sync** — cursor updates visible to remote user within 200ms
- **Bundle size** — main bundle under 200kb gzipped; per-layer code splitting enforced
- **Mobile** — all above targets must hold on a mid-range Android device (not just desktop)